

Team ESCOM-IPN at CLEF 2026: Multimodal Financial Decision Making using LLMs, Chronos, and Auto-Reflective Prompt Routing

Rodrigo¹, Jona¹, Baru¹, and David¹

¹Escuela Superior de Cómputo (ESCOM) - Instituto Politécnico Nacional (IPN),
Mexico City, Mexico

Abstract

This Working Note describes the participation of the ESCOM-IPN team in Task 3 of the CLEF 2026 FinMMEval lab. We propose a highly structured, multimodal retrieval-augmented generation (RAG) pipeline to predict financial trading actions (BUY, HOLD, SELL) across diverse assets. Our system integrates unstructured news and SEC filings with numerical scalars and temporal price sequences. We introduce a novel dual-strategy *Prompt-Builder* featuring a tripartite temporal memory (rolling buffer, self-reflection, and top-K retrieval), allowing Large Language Models (LLMs) to reason over current market contexts while continuously learning from empirical trading results. Furthermore, we implement *finvert*, a highly optimized FinBERT-based module for robust market sentiment extraction. Our methodology demonstrates how programmatic prompt engineering, heuristic risk management, and chronological FIFO queuing of semantic summaries can significantly enhance LLM reasoning in simulated financial environments.

Keywords: Financial Forecasting, Large Language Models, Sentiment Analysis, Prompt Engineering, Temporal Memory, FinMMEval, CLEF 2026.

1 Introduction

Financial decision-making requires the rapid synthesis of heterogeneous data sources. In the CLEF 2026 FinMMEval Lab (Task 3), participants are challenged to develop agents capable of processing numerical scalars, temporal price histories, and textual news to recommend trading actions (BUY, HOLD, SELL) for assets such as Bitcoin, Tesla, and Apple.

While Large Language Models (LLMs) excel at natural language understanding, they traditionally struggle with raw numerical sequences and lack inherent contextual memory of past actions in simulated trading environments. To bridge this gap, our team developed a comprehensive multimodal pipeline.

The core contribution of our work lies in the intelligent orchestration of data prior to LLM inference. Specifically, we detail two critical components designed by our team: (1) **finvert**, an automated and resource-optimized sentiment extraction module leveraging FinBERT, and (2) the **PromptBuilder**, a sophisticated prompt engineering engine equipped with a tripartite temporal memory. By providing the LLM not just with current data, but with a rolling history of its past decisions and their resulting financial rewards, we simulate a realistic trading agent that adapts to market dynamics and mitigates risk through self-reflection.

2 System Architecture and Dataset Formulation

[Notau: Expandir esta sección. Describir script 'generate_all.py', cómo se construyó 'finmeval_raw.csv' mezclando HuggingFace y Yahoo Finance, y los detalles de Chronos Encoder.]

The overall architecture transforms raw multimodal data into structured prompts for LLM inference. The pipeline operates as follows:

1. **Data Ingestion:** Raw data from CLEF and Yahoo Finance is collected and unified, encompassing textual news, 10-K/10-Q filings, and technical scalars.
2. **Temporal Encoding (Chronos):** Time-series data is processed to extract temporal embeddings. [David: Add Chronos details here].
3. **Sentiment Extraction (finvert):** Textual data is classified to extract sentiment probabilities.
4. **Prompt Orchestration:** The *PromptBuilder* fuses all signals alongside the agent’s trading memory.
5. **LLM Decision Making:** The final prompt is submitted to the decision LLM to output a structured JSON containing the action and rationale.

3 Temporal Encoding: The Chronos-Bolt Module

To capture the predictive signal embedded in historical price trajectories, our pipeline employs **Chronos-Bolt** (`amazon/chronos-bolt-small`), a transformer-based probabilistic forecasting model pre-trained on large collections of real-world time series. Unlike traditional statistical approaches, Chronos-Bolt learns a distribution over future values, enabling the extraction of uncertainty-aware forecasts directly from raw numerical sequences.

The module is loaded via the `BaseChronosPipeline` abstraction, which handles device placement automatically. When a CUDA-capable GPU is available, the model is instantiated in `bfloat16` precision to reduce VRAM consumption while preserving numerical stability. In CPU-only environments, standard floating-point inference is applied as a fallback.

3.1 Input Formulation and Batch Processing

Each asset’s historical prices are stored per-day as a serialized list in the `prices_sequence` column of the dataset. Prior to inference, each entry is deserialized into a typed list of floats, forming the context window fed to the model.

To maximize GPU throughput, sequences are grouped into batches of 16 and converted to PyTorch tensors. For each batch, the model is queried with `prediction_length=1`, restricting the forecast horizon to a single future time step — the next trading day. This deliberate constraint enforces that the model can only reason over observed history, preserving temporal isolation and avoiding any form of look-ahead bias.

3.2 Forecast Extraction and Direction Classification

Chronos-Bolt outputs a probabilistic forecast tensor of shape $(\text{batch} \times \text{num_samples} \times \text{prediction_length})$, where the `num_samples` dimension represents draws from the learned predictive distribution. To obtain a single point estimate per sequence, we compute the **median** across the sample dimension:

$$\hat{p}_{t+1} = \text{median}_s F_s(x_{1:t}) \quad (1)$$

where F_s denotes the s -th sample of the forecast distribution conditioned on the observed price history $x_{1:t}$. The resulting scalar $\hat{p}_{t+1}$ is then converted into a discrete directional class by computing the relative price change with respect to the last observed price p_t :

$$\Delta = \frac{\hat{p}_{t+1} - p_t}{p_t} \quad (2)$$

A symmetric threshold $\tau = 0.001$ ($\pm 0.1\%$) governs the classification into three mutually exclusive categories:

$$\text{class} = \begin{cases} 1 & \text{if } \Delta > \tau \quad (\text{UP}) \\ 2 & \text{if } \Delta < -\tau \quad (\text{DOWN}) \\ 0 & \text{otherwise} \quad (\text{HOLD}) \end{cases} \quad (3)$$

This threshold was selected to filter out negligible price fluctuations below the level of market noise, ensuring that the HOLD class captures genuine price stability rather than prediction imprecision.

3.3 Evaluation and Output

Upon completing inference over the full dataset, the module computes both global and per-asset evaluation metrics. Global performance is assessed via accuracy, macro-averaged F1, and weighted F1 scores, while per-asset metrics allow identification of assets whose temporal price patterns are more or less amenable to the model’s forecasting strategy. The complete predictions and metrics are persisted to `predictions.csv` and `metrics.csv` respectively, serving as structured inputs to subsequent stages of the pipeline.

4 Multimodal Sentiment Extraction: The *finvert* Module

The *finvert* module acts as the market sentiment sensor of our architecture. Its primary objective is to process unstructured financial texts and quantify them into actionable numerical probabilities.

4.1 Auto-Discovery and Text Combination

Financial sentiment is rarely derived from a single source. Our system aggregates three primary textual streams per day in a strict priority hierarchy: daily news, annual reports (10-K), and quarterly reports (10-Q).

To optimize computational resources, the module implements an auto-discovery and filtering mechanism (`has_real_text`). Datasets lacking actual textual data (e.g., pure price-action assets from Yahoo Finance flagged as `[NO.TEXT]`) are automatically bypassed. These rows are deterministically assigned a *neutral* label with a 1.0 confidence score without invoking the GPU, vastly improving pipeline efficiency and preventing unnecessary VRAM allocation.

4.2 FinBERT Inference and Batch Processing

For valid textual entries, we utilize *FinBERT* (ProsusAI/finbert), a BERT model pre-trained on large-scale financial corpora. Texts are processed in batches of 32 to maximize GPU throughput.

The module tokenizes the combined daily text (truncated to a maximum length of 512 tokens) and outputs logits through a 3-class classification head. Applying a softmax function, we extract the probabilities for each class:

$$P(C_i|x) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}} \quad (4)$$

where $C \in \{\text{positive, negative, neutral}\}$. The output is stored in a structured format containing the `sentiment_label`, the absolute `sentiment_score`, and the exact probability distribution. This granular output prevents the downstream LLM from interpreting a weak positive signal with the same weight as a strong one.

5 Dynamic Prompt Routing and Temporal Memory

The *PromptBuilder* is the central nervous system of our pipeline, encompassing over 600 lines of orchestration logic. Its responsibility is to synthesize sentiment analysis, technical scalars, portfolio status, and temporal sequences into a highly structured prompt.

Crucially, to simulate a realistic trading environment and prevent look-ahead bias, we enforce **strict temporal isolation**: the prompt for day T is generated entirely *before* the portfolio memory is updated with the market outcomes of day T .

5.1 Tripartite Memory Architecture

At any time step t , the state of the market is captured by a `SensorOutputs` object. Meanwhile, the system maintains a `MemoryContext` of `DayRecord` objects. Inspired by recent advancements in LLM trading agents (such as FLAG-TRADER and FINMEM), we implemented three types of memory:

- **Rolling Buffer**: A sequential tracker of recent actions and rewards.
- **Self-Reflection**: Analysis of past errors and successes.
- **Top-K Retrieval**: Extraction of the most relevant historical trades.

5.2 Strategy V1: Programmatic Auto-Reflection

Strategy V1 is a deterministic, highly efficient prompt routing mechanism requiring no auxiliary API calls. It programmatically translates numerical scalars (e.g., RSI, portfolio exposure) and categorical data into natural language.

The primary innovation in V1 is the **Auto-Reflection Summary**. The module identifies the most successful and most detrimental trades within a historical window. The generated prompt presents the LLM with an ASCII-formatted rolling history table and a specific reflection block:

*"CASOS EXITOSOS RECIENTES: 2024-01-15: Accion=BUY, Reward=+0.0234...
CASOS NEGATIVOS RECIENTES: 2024-01-17: Accion=SELL, Reward=-0.0120..."*

This structural constraint forces the LLM to ground its current decision by learning from empirical past mistakes, effectively mimicking human reinforcement learning.

5.3 Strategy V2: Dual-LLM Pipeline and Semantic FIFO Queuing

To further enhance reasoning, Strategy V2 employs a chained dual-LLM architecture, operating as follows:

1. **LLM-1 (Summarizer)**: A lightweight model (e.g., GPT-4o-mini or Llama-3.2-3B) reads the raw data of past days and translates the events into a rich, narrative summary. To ensure deterministic testing during development, we also implemented a `StubClient` factory pattern.

2. **FIFO Queuing & Caching:** These narrative summaries are pushed into a chronological First-In-First-Out (FIFO) queue. To heavily optimize API costs and reduce latency, outputs from LLM-1 are cached directly in the `DayRecord.llm_summary` attribute.
3. **LLM-2 (Decision):** A heavier reasoning model (e.g., GPT-4o) receives the entire narrative FIFO queue alongside the current market context to make the final BUY/HOLD/SELL decision.

5.4 Heuristic Risk Management via System Prompts

Both strategies are governed by a strict *System Prompt* that encodes risk-management heuristics. For instance, the LLM is explicitly instructed to prefer SELL or HOLD if the portfolio exposure exceeds 80%, to avoid BUY recommendations if cash is depleted, and to flag high volatility coupled with negative sentiment as extreme risk.

6 Model Optimization and Training Setup

[Nota: Esta es su sección. Detallen el SFT Warm-up (1 época, 30 min) para enseñar el formato XML, la optimización Optuna TPE (3 trials, seed 42), y el entrenamiento GRPO (6 min). No olvidar explicar la Reward Function asimétrica 3×3 y cómo la aumentaron a -2.0 para el HOLD para romper el equilibrio de Nash.]

7 Results and Validation

[Nota: Documenten aquí el contraste entre el Fast Mode y la Validación Completa (896 filas). Hablen del modelo ganador (exp_20260506_085649_254046) y cómo se seleccionó manualmente por su Cumulative Return (90.67%) a pesar de tener un MDD alto, contrastándolo con el modelo de mejor Fitness (1.974). Usen la tabla a continuación como base para sus datos.]

Model ID	Fitness	Cumulative Return	Sharpe Ratio	MDD
exp_085649 (Selected)	0.1069	90.67%	2.9117	48.05%
exp_111504 (Best Fit)	1.9740	19.34%	1.9740	17.48%
exp_004508 (Best SR)	-0.0759	89.76%	3.0408	51.17%

Table 1: Full validation results comparing the selected model against other Optuna trials.

8 Conclusion and Future Work

In this Working Note, we presented a comprehensive RAG pipeline for financial decision-making in the CLEF 2026 FinMMEval Lab. By developing the *finvert* module for precise sentiment extraction and a novel *PromptBuilder* equipped with a tripartite temporal memory, we successfully bridged the gap between raw quantitative market data and qualitative LLM reasoning.

Our dual-strategy prompt orchestration demonstrated that providing an LLM with a structured, self-reflective memory of its past actions significantly enhances its ability to manage risk and make context-aware trading decisions.

Future work will focus on [agregar sus puntos sobre mejorar el split temporal (Walk-forward validation), balancear el dataset entre equity/crypto, y automatizar la selección por fitness en lugar de selección manual por CR.].

Acknowledgments

We extend our gratitude to the Artificial Intelligence Club (GAIA) at ESCOM-IPN for their continuous support and computational resources.